

El *forward* completo del *pipeline* CRB

De la simulación del transitorio a las elipses de incertidumbre,
con la justificación de cada paso

Repositorio `crb` — rama consolidada

3 de septiembre de 2026

Resumen

Estas notas explican, de principio a fin y sin saltos, todo el *forward* del *pipeline* actual del repositorio: el simulador vectorizado en PyTorch (`simulator.py`), el modelo estadístico de Poisson que se le acopla, el jacobiano por diferencias finitas, la información de Fisher, la cota de Cramér–Rao y las elipses de incertidumbre que se dibujan al final, con la grilla de referencia de 32×32 píxeles. Cada fórmula va seguida de su justificación: por qué ese término, por qué esa decisión de implementación y qué pasaría si no estuviera. Todas las expresiones se han verificado contra el código, y cada paso cita el archivo y la función donde vive.

Cuatro resultados de la verificación merecen adelantarse, porque corrigen ideas que circulan sobre este código. Primero, `facet.obj` es un cuadrilátero *plano* de dos triángulos con extensión nula en z , de modo que la rama $1,1/\text{extents}_z$ del factor de escala **nunca se activa** y la faceta acaba midiendo $0,5 \times 1,585$ m. Segundo, la densificación no deja 10 000 triángulos sino **32 768**, porque 10 000 es un piso y cada subdivisión cuadruplica. Tercero, el *pitch* de 1,57 rad no es exactamente $\pi/2$, así que la normal de la faceta queda inclinada $0,0456^\circ$ respecto de la horizontal. Y cuarto, el `np.roll` final es una permutación cíclica de los píxeles y por tanto **deja la información de Fisher exactamente invariante**: se demuestra en la proposición 2 y se comprueba numéricamente a precisión de máquina.

Índice

1. Planteamiento y notación	2
1.1. La escena	2
1.2. Los parámetros a estimar	3
1.3. Qué queremos calcular	3
2. La simulación, paso a paso	4
2.1. Colocación de la faceta	4
2.2. Las dos distancias y el modelo de dos rebotes	6
2.3. Los cuatro cosenos	7
2.4. La atenuación $1/(4\pi^2 d_1^2 d_2^2)$	9
2.5. Área del triángulo e intensidad del láser	9
2.6. Oclusión: derivación de x_{int}	9
2.7. Cuantización temporal y depósito	11
2.8. El transitorio esperado	13
3. Del transitorio al modelo estadístico	13
3.1. Por qué Poisson	13
3.2. El fondo b	14
3.3. El CRB no consume datos	15
3.4. Independencia entre índices	15

4. La información de Fisher	15
4.1. Derivación	15
4.2. Cómo leerla	16
5. El jacobiano por diferencias finitas	16
5.1. Diferencias centrales	16
5.2. El problema honesto: el <i>forward</i> es escalonado	16
5.3. Por qué funciona a pesar de todo	17
5.4. Por qué <code>float64</code>	17
5.5. El adaptador de dos a tres valores	19
6. La cota de Cramér–Rao	19
6.1. El enunciado y su alcance	19
6.2. La correlación entre ρ y φ	20
7. De la matriz a la elipse	21
7.1. La región de confianza es una elipse	21
7.2. Por qué $k = 3$	21
7.3. La inclinación <i>es</i> la correlación	22
7.4. Por qué se dibuja en el espacio de parámetros	22
8. El <i>baseline</i> de 32×32	23
8.1. Cómo escala la precisión con la resolución	23
8.2. La rejilla <i>baseline</i>	23
8.3. Cómo leer la rejilla	25
9. Mapa del código	25
10. Limitaciones y salvedades	26

1. Planteamiento y notación

1.1. La escena

La geometría es la de una *cámara de esquina*. Todo vive en un sistema de coordenadas con el suelo en $z = 0$:

- Un **láser** pulsado está en el origen $\mathbf{p}_\ell = (0, 0, 0)$, apuntando hacia arriba con normal $\mathbf{n}_\ell = (0, 0, 1)$ (`laser_pos`, `laser_normal` en `simulator.simulation`). El punto del suelo que ilumina se comporta como un emisor difuso secundario.
- Una **arista ocluyente** coincide con la recta $y = 0$: el semiplano $x < 0$ está tapado por la pared de la esquina y el semiplano $x > 0$ es el hueco por el que la luz puede pasar. Esta convención no está escrita como una geometría explícita en el código, sino *implícita* en el test de oclusión de la sección 2.6.
- Un **sensor SPAD** observa un parche del suelo: un cuadrado de `camera_FOV`= 0,25 m de lado, centrado en `camera_FOV_center`= $(0, -\text{FOV}/2, 0) = (0, -0,125, 0)$, dividido en $N \times N$ píxeles con $N = \text{cam_pixel_dim} = 32$.
- Una **faceta oculta** (`facet.obj`) de anchura $w = 0,5$ m está detrás de la arista, en el lado $y > 0$.

Los centros de píxel se construyen en `simulate_transient_torch` con dos `linspace` que dejan medio píxel de margen en cada borde:

$$x_j = -\frac{\text{FOV}}{2} + \frac{\text{FOV}}{2N} + j \frac{\text{FOV}}{N}, \quad y_i = -\text{FOV} + \frac{\text{FOV}}{2N} + i \frac{\text{FOV}}{N}, \quad i, j = 0, \dots, N-1. \quad (1)$$

El *pitch* de píxel es por tanto

$$\Delta_{\text{px}} = \frac{\text{FOV}}{N} = \frac{0,25 \text{ m}}{32} = 7,8125 \text{ mm}. \quad (2)$$

Observación 1 (El *pitch* de la grilla *baseline* es 7,81 mm, no 3,9 mm). Es fácil confundirse aquí, porque 3,9 aparece dos veces en la configuración por motivos distintos: 3,9 mm era el *pitch* de la grilla *antigua* de 64×64 ($0,25/64$), y $\Delta t = 3,9 \cdot 10^{-10} \text{ s}$ es el tamaño de *bin* temporal. Con la grilla *baseline* de 32×32 el *pitch* es el doble, 7,8125 mm. La coincidencia numérica es casualidad.

El eje temporal se discretiza en *bins* de $\Delta t = \text{bin_size} = 3,9 \cdot 10^{-10} \text{ s}$. Lo que importa es la longitud de camino que cabe en un *bin*:

$$c \Delta t = 299\,792\,458 \frac{\text{m}}{\text{s}} \times 3,9 \cdot 10^{-10} \text{ s} = 0,116919 \text{ m} \approx 11,7 \text{ cm de camino recorrido}. \quad (3)$$

Esta cifra reaparecerá constantemente: es la resolución bruta del sensor y la fuente del principal problema numérico del *pipeline* (sección 5).

1.2. Los parámetros a estimar

La pose de la faceta se describe en coordenadas polares sobre el suelo por

$$\psi = (\rho, \varphi), \quad \text{posición} = (\rho \cos \varphi, \rho \sin \varphi, 0), \quad (4)$$

y la faceta se coloca de pie, con la normal apuntando hacia el origen dentro del plano horizontal:

$$\mathbf{n}(\varphi) = (-\cos \varphi, -\sin \varphi, 0). \quad (5)$$

La sección 2.1 demuestra que la cadena de rotaciones del código produce exactamente esta normal.

Observación 2 (Por qué polares y no cartesianas). La geometría del problema es radial: el láser está en el origen y la arista ocluyente pasa por él. El tiempo de vuelo depende esencialmente de ρ (la distancia) y la oclusión depende esencialmente de φ (el ángulo respecto del hueco). Separar el problema en «cuán lejos» y «en qué dirección» desacopla los dos mecanismos físicos de información, y hace que las elipses de la sección 7 se lean directamente como «incertidumbre en rango» frente a «incertidumbre en azimut».

1.3. Qué queremos calcular

Con la faceta en ψ , el simulador produce un transitorio esperado $\mathbf{y}(\psi) \in \mathbb{R}^{N \times N \times T}$. Un experimento real mediría un transitorio *ruidoso*, y de él se querría estimar $\hat{\psi}$. La pregunta que responden estas notas no es *cómo* estimar, sino **cuál es la mejor precisión alcanzable**: la cota de Cramér–Rao (CRB), que acota por debajo la covarianza de cualquier estimador insesgado. El recorrido es

$$\underbrace{\psi}_{\S 2} \longrightarrow \underbrace{\mathbf{y}(\psi)}_{\S 3} \longrightarrow \underbrace{\mathbf{g} = \mathbf{y} + b}_{\S 3} \longrightarrow \underbrace{\mathbf{J} = \partial \mathbf{g} / \partial \psi}_{\S 5} \longrightarrow \underbrace{\mathbf{I} = \mathbf{J}^\top \text{diag}(1/g) \mathbf{J}}_{\S 4} \longrightarrow \underbrace{\mathbf{C} = \mathbf{I}^{-1}}_{\S 6} \longrightarrow \underbrace{\text{elipse}}_{\S 7}.$$

La figura 1 resume la escena y adelanta el test de oclusión.

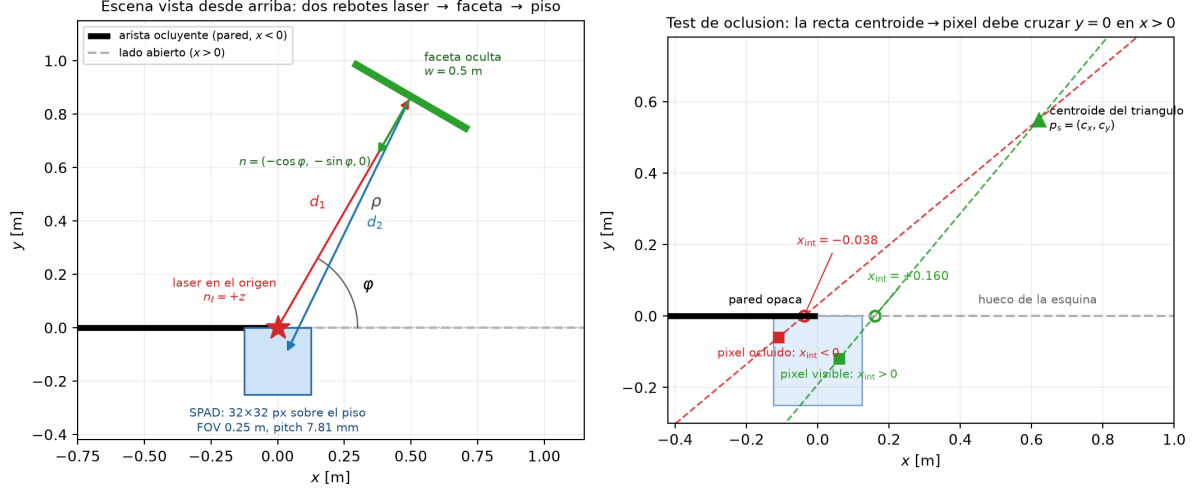


Figura 1: Izquierda: la escena vista desde arriba. El láser en el origen ilumina el suelo; la luz difundida llega a la faceta oculta (recorrido d_1), se difunde de nuevo y alcanza el parche de suelo que el SPAD observa (recorrido d_2). La arista ocluyente tapa $x < 0$. Derecha: el test de oclusión de la sección 2.6: la recta que une el centroide de un triángulo con un píxel debe cruzar $y = 0$ del lado abierto, $x_{\text{int}} > 0$.

2. La simulación, paso a paso

Toda esta sección describe `simulator.py`. La malla se coloca en `_transform_object_mesh`, el bucle radiométrico vive en `simulate_transient_torch` y `simulation` orquesta a los dos.

2.1. Colocación de la faceta

Carga y densificación. El archivo `objects/facet.obj` es, medido directamente, un **cuadrilátero plano** de 2 triángulos y 4 vértices, con extensiones (0,390656, 1,238402, 0) m. A continuación

```
obj = densify_mesh_if_needed(obj, min_triangles=10000)
```

subdivide la malla. La subdivisión de `trimesh` parte cada triángulo en cuatro, de modo que el número de triángulos sigue $2 \cdot 4^k$; el primer k con $2 \cdot 4^k \geq 10000$ es $k = 7$, y por tanto

$$T_{\Delta} = 2 \cdot 4^7 = 32768 \text{ triángulos.} \quad (6)$$

Observación 3 (No son 10 000 triángulos, son 32 768). `min_triangles` es un *piso*, no un objetivo: la subdivisión no puede aterrizar en 10 000 porque solo sabe cuadruplicar. El documento y cualquier cálculo de coste deben usar 32 768.

Por qué densificar: la suma es una cuadratura. Esta es la justificación central de todo el modelo. La energía que la faceta reenvía hacia un píxel es una **integral de superficie** sobre la faceta S ,

$$y_{\text{px}} = \int_S f(\mathbf{p}_s) dA(\mathbf{p}_s), \quad (7)$$

con f el integrando radiométrico que se construye en las secciones siguientes. El simulador la aproxima por una **regla del punto medio** por triángulo: evalúa f en el centroide $\mathbf{c}_t$ y la pesa por el área A_t ,

$$y_{\text{px}} \approx \sum_{t=1}^{T_{\Delta}} f(\mathbf{c}_t) A_t. \quad (8)$$

El error de esta cuadratura decrece al refinar la malla, y hace falta refinar mucho por dos razones: f contiene $1/d_2^2$ y cosenos, que son no lineales sobre una faceta de más de un metro y medio de alto; y sobre todo f contiene un **indicador de oclusión discontinuo**, de modo que con pocos triángulos la visibilidad parcial de la faceta se representa muy groseramente. Si no se densificara (2 triángulos), la faceta sería visible o invisible casi en bloque; con 32 768 triángulos la frontera de la sombra queda resuelta finamente. En la sección 5 veremos que esta misma densificación es lo que hace viable el jacobiano por diferencias finitas.

Escala. El código construye una lista de factores candidatos y toma el menor:

$$s = \min \left(\underbrace{\frac{w}{\text{extents}_x}}_{\text{si } \text{extents}_x > 10^{-12}}, \underbrace{\frac{1,1}{\text{extents}_z}}_{\text{si } \text{extents}_z > 10^{-12}} \right). \quad (9)$$

Con `facet.obj` se tiene $\text{extents}_z = 0$ exactamente (es un cuadrilátero plano en el plano xy), así que la guarda $\text{extents}_z > 10^{-12}$ **descarta el segundo candidato** y sobra únicamente el primero:

$$s = \frac{w}{\text{extents}_x} = \frac{0,5}{0,390656} = 1,279898. \quad (10)$$

Observación 4 (La cota de 1,1 m de alto es inerte para esta faceta). Es tentador leer (9) como «la faceta no pasará de 1,1 m». Con `facet.obj` eso es falso: como la malla es plana, la rama de z no participa, y la faceta escalada acaba midiendo

$$0,5 \text{ m (ancho)} \times 1,585 \text{ m (alto)}, \quad \text{área} = 0,79251 \text{ m}^2,$$

es decir, $1,585 = 1,238402 \times 1,279898$, bastante por encima de 1,1. La cota solo actuaría con un objeto que tuviera espesor en z *antes* de las rotaciones. Este detalle importa porque el alto de la faceta es lo que dispersa los tiempos de vuelo (sección 2.7).

Rotaciones. Se aplican en este orden: *pitch* de 1,57 rad alrededor de x , *roll* de 0 alrededor de y , y *yaw* de $\theta = \varphi + 3\pi/2$ alrededor de z . El *pitch* pone de pie el cuadrilátero, que originalmente yace en el plano xy con normal $\mathbf{n}_0 = (0, 0, 1)$:

$$\mathbf{R}_x(1,57) \mathbf{n}_0 = (0, -\sin 1,57, \cos 1,57) = (0, -0,99999968, 0,00079633). \quad (11)$$

Observación 5 ($1,57 \neq \pi/2$: la faceta queda inclinada $0,0456^\circ$). Como $\pi/2 = 1,5707963\dots$, el *pitch* de 1,57 deja un residuo $\pi/2 - 1,57 = 7,96 \cdot 10^{-4}$ rad. La normal conserva una componente z de $\cos(1,57) = 7,9633 \cdot 10^{-4}$, lo que equivale a $0,0456^\circ$ de desviación respecto de la normal horizontal ideal (5). Es un detalle inocuo para el CRB (afecta a los cosenos en el quinto decimal) pero conviene no presentar 1,57 como si fuese $\pi/2$ exacto: es un valor *redondeado* heredado de la configuración original.

Proposición 1 (El *yaw* $\theta = \varphi + 3\pi/2$ produce la normal (5)). Sea $\mathbf{n}_1 = (0, -1, 0)$ la normal tras un *pitch* de exactamente $\pi/2$. Entonces $\mathbf{R}_z(\varphi + 3\pi/2) \mathbf{n}_1 = (-\cos \varphi, -\sin \varphi, 0)$.

Demostración. Con $\mathbf{R}_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$ se tiene $\mathbf{R}_z(\theta)(0, -1, 0)^\top = (\sin \theta, -\cos \theta, 0)^\top$. Sustituyendo $\theta = \varphi + 3\pi/2$ y usando $\sin(\varphi + 3\pi/2) = -\cos \varphi$ y $\cos(\varphi + 3\pi/2) = \sin \varphi$,

$$\mathbf{R}_z(\varphi + \frac{3\pi}{2})(0, -1, 0)^\top = (-\cos \varphi, -\sin \varphi, 0)^\top. \quad \square$$

El desplazamiento de $3\pi/2$ es, entonces, exactamente el necesario para que *la cara de la faceta mire al origen*, que es lo que interesa: la faceta debe recibir luz del punto iluminado del suelo y reenviarla, no darle la espalda. Verificado numéricamente sobre la malla real, la normal media ponderada por área da $(-0,5, -0,866025, 0,000796)$ para $\varphi = 60^\circ$, a $0,0456^\circ$ de la predicción (5) — exactamente el residuo de la observación 5.

Apoyo en el suelo y traslación. Finalmente

```
obj.apply_translation([0,0,-z_min])    y luego    obj.apply_translation(v1)
```

con $\mathbf{v1} = (\rho \cos \varphi, \rho \sin \varphi, 0)$. El primer paso baja la malla hasta que su vértice más bajo toca $z = 0$, y el segundo la lleva a la pose pedida. El orden importa: primero se apoya en el suelo alrededor del origen y después se traslada, de forma que la faceta **siempre queda de pie sobre el suelo** independientemente de ρ y φ . Su centroide ponderado por área queda en $z = 0,792514$ m, la mitad de 1,585, como corresponde a un rectángulo uniforme.

2.2. Las dos distancias y el modelo de dos rebotes

Para cada triángulo con centroide $\mathbf{c}_t$ y cada píxel en $\mathbf{q}_p$:

$$d_1(t) = \|\mathbf{p}_\ell - \mathbf{c}_t\| \quad \text{lps} = \text{laser_pos} - \text{centers}, \text{d1s} = \text{sum}(\text{lps} * \text{lps}), \quad (12)$$

$$d_2(t, p) = \|\mathbf{q}_p - \mathbf{c}_t\| \quad \text{fovsp} = \text{cam_pos} - \text{centers}, \text{d2s} = \text{sum}(\text{fovsp} * \text{fovsp}). \quad (13)$$

Nótese que d_1 depende solo del triángulo (vector de tamaño T_Δ) mientras que d_2 depende del par (vector $T_\Delta \times P$); el código explota esa asimetría con *broadcasting*, lo que ahorra memoria.

Por qué dos rebotes y no tres. El camino modelado es

$$\text{láser} \xrightarrow{d_1} \text{faceta oculta} \xrightarrow{d_2} \text{parche de suelo} \xrightarrow{\text{óptica}} \text{SPAD}.$$

Hay dos *reflexiones difusas* en el trayecto: la del punto iluminado del suelo (que reemite hacia la faceta) y la de la faceta (que reemite hacia el suelo observado). El último tramo, del parche de suelo al sensor, **no es una reflexión**: el SPAD *mira* directamente ese parche, así que cada píxel recoge la radiancia del punto del suelo que le corresponde. Por eso no aparece ni un tercer factor $1/d^2$ ni un quinto coseno. Esta es la diferencia fundamental con el planteamiento del *writeup* de referencia, que sí incluye el tramo cámara–suelo y en consecuencia arrastra una constante distinta (sección 2.4).

Los rebotes de orden superior (luz que va y vuelve varias veces) se desprecian: cada reflexión difusa cuesta un factor $\sim 1/(\pi d^2)$, y a distancias del orden del metro eso son dos órdenes de magnitud por rebote. Es la aproximación estándar en NLOS de tres rebotes, aquí con el tercero absorbido por la óptica del sensor.

La figura 2 muestra d_2 , la atenuación y el camino total sobre la grilla de píxeles.

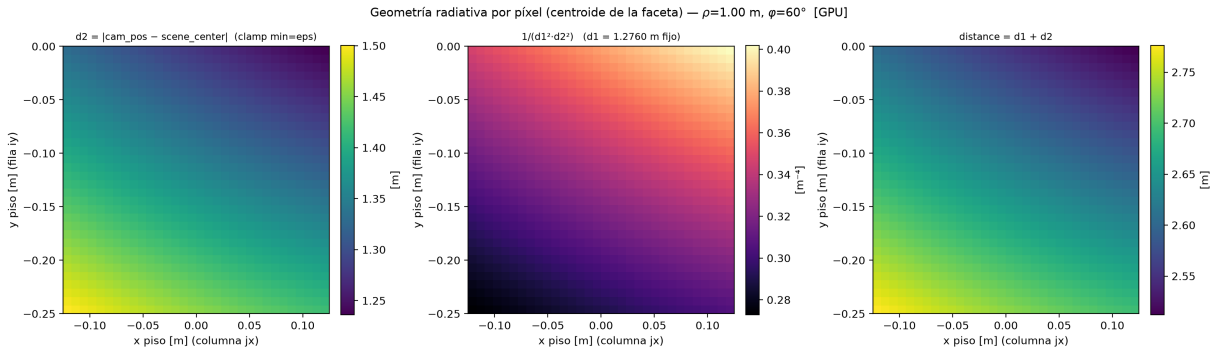


Figura 2: Geometría radiativa por píxel para el centroide de la faceta en $(\rho, \varphi) = (1 \text{ m}, 60^\circ)$. Izquierda: d_2 , que varía solo entre 1,24 y 1,50 m sobre todo el FOV. Centro: la atenuación $1/(d_1^2 d_2^2)$ con $d_1 = 1,276$ m fijo. Derecha: el camino total $d_1 + d_2$, que es lo que se cuantiza en *bins*. Generada por `plot_simulator_components.py -backend gpu`.

2.3. Los cuatro cosenos

Cada reflexión difusa obedece la ley de Lambert: la potencia que una superficie emite o recibe en una dirección se reduce por el coseno del ángulo entre esa dirección y la normal, porque lo que cuenta es el *área proyectada* perpendicular al haz. En un trayecto de dos rebotes hay cuatro eventos de este tipo — una emisión y una incidencia en cada una de las dos superficies — y el código los calcula exactamente así:

$$\begin{aligned}
 \text{dot3} &= \max\left(0, \left\langle \mathbf{n}_\ell, \frac{\mathbf{c}_t - \mathbf{p}_\ell}{d_1} \right\rangle\right) && \text{emisión en el punto iluminado del suelo,} \\
 \text{dot1} &= \max\left(0, \left\langle \mathbf{n}_t, \frac{\mathbf{p}_\ell - \mathbf{c}_t}{d_1} \right\rangle\right) && \text{incidencia en la faceta,} \\
 \text{dot2} &= \max\left(0, \left\langle \mathbf{n}_t, \frac{\mathbf{q}_p - \mathbf{c}_t}{d_2} \right\rangle\right) && \text{emisión desde la faceta,} \\
 \text{dot4} &= \max\left(0, \left\langle \mathbf{n}_{\text{piso}}, \frac{\mathbf{c}_t - \mathbf{q}_p}{d_2} \right\rangle\right) && \text{incidencia en el parche de suelo,}
 \end{aligned} \tag{14}$$

con $\mathbf{n}_{\text{piso}} = (0, 0, 1)$ (`floor_normal`).

Correspondencia exacta con el código, incluidos los signos. Merece la pena seguirlos uno a uno, porque los signos son fáciles de confundir. El código define `lps = laser_pos - centers`, es decir $\mathbf{p}_\ell - \mathbf{c}_t$, y `fovsp = cam_pos - centers`, es decir $\mathbf{q}_p - \mathbf{c}_t$. Entonces:

- `dot1 = clamp(sum(normals * (lps/d1)))`: usa `+lps`, o sea la dirección *de la faceta hacia el láser*. Correcto para una incidencia: se mide el ángulo entre la normal de la faceta y la dirección de donde viene la luz.
- `dot3 = clamp(sum(laser_normal * (-lps/d1)))`: usa `-lps =  $\mathbf{c}_t - \mathbf{p}_\ell$` , la dirección *del láser hacia la faceta*. Correcto para una emisión: se mide el ángulo entre la normal del emisor y la dirección de salida.
- `dot2 = clamp(sum(normals * (fovsp/d2)))`: usa `+fovsp`, dirección *de la faceta hacia el píxel*: emisión desde la faceta.
- `dot4 = clamp(sum(floor_normal * (-fovsp/d2)))`: usa `-fovsp =  $\mathbf{c}_t - \mathbf{q}_p$` , dirección *del píxel hacia la faceta*: incidencia sobre el suelo.

El patrón es consistente: en las *emisiones* el vector apunta *alejándose* del emisor, y en las *incidencias* apunta *hacia* la fuente. Los dos que usan el signo negativo son precisamente los dos que se miden contra una normal que no es la de la faceta.

Por qué el `clamp`. `max(0, ·)` implementa el hecho físico de que **una cara no irradia hacia atrás ni recibe luz por detrás**. Si $\langle \mathbf{n}, \mathbf{u} \rangle < 0$, la dirección $\mathbf{u}$ está en el hemisferio opuesto a la cara y la contribución debe ser cero, no negativa. Sin el `clamp` los triángulos orientados al revés restarían energía y podrían producir intensidades negativas, que además romperían el modelo de Poisson de la sección 3 (una tasa no puede ser negativa).

En la pose de la figura 3 el `clamp` está **inactivo** (los mínimos sin recortar son 0,762 para `dot2` y 0,528 para `dot4`, ambos positivos): con la faceta mirando al origen y el FOV pequeño, ningún píxel cae detrás de la faceta. El `clamp` es una guarda que solo se activa en poses extremas, pero su presencia es la razón por la que el *forward* es no diferenciable en un conjunto de medida no nula de configuraciones (sección 5).

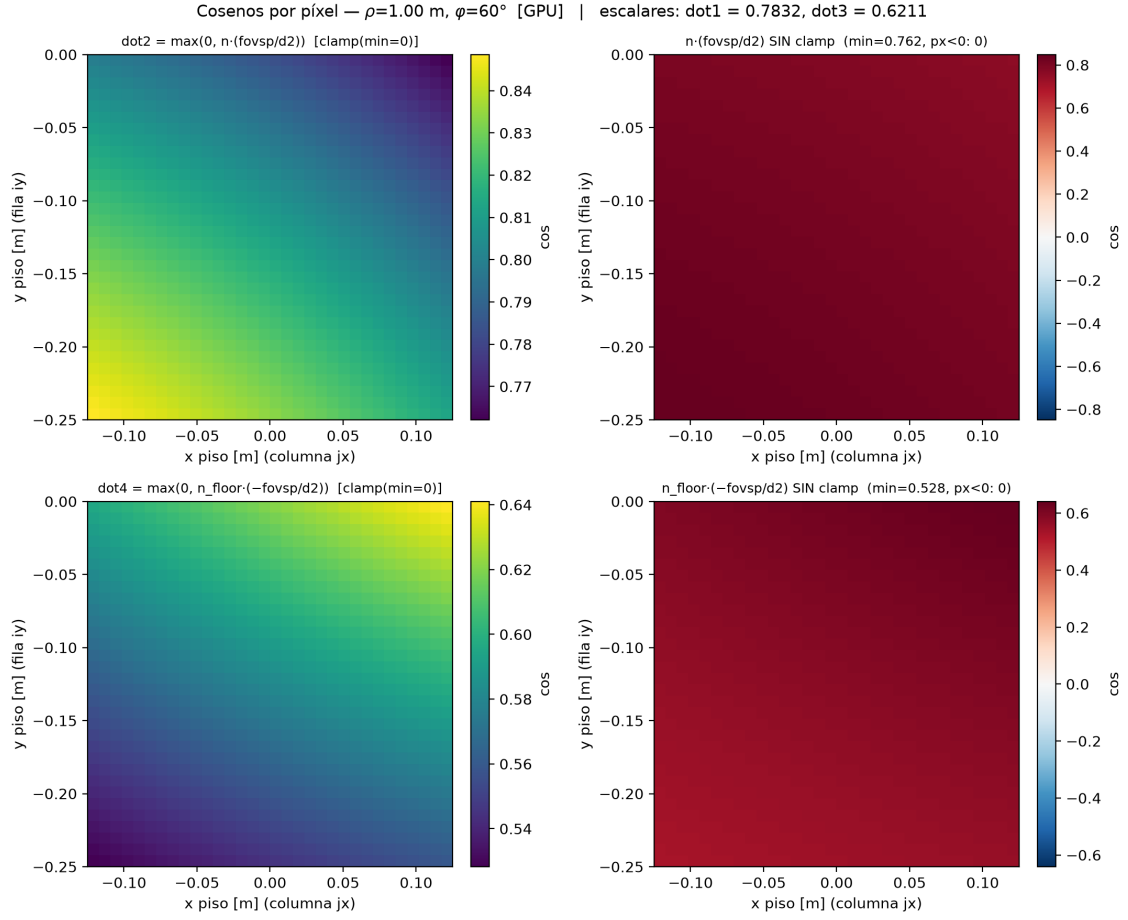


Figura 3: Los cosenos dependientes del píxel, dot2 y dot4 , con y sin `clamp`, para $(\rho, \varphi) = (1 \text{ m}, 60^\circ)$. Los escalares que solo dependen del triángulo valen $\text{dot1} = 0,7832$ y $\text{dot3} = 0,6211$. Las columnas de la derecha muestran que en esta pose no hay ningún píxel con coseno negativo: el `clamp` no actúa.

2.4. La atenuación $1/(4\pi^2 d_1^2 d_2^2)$

El código escribe la intensidad como

$$I_{t,p} = \frac{I_{\text{láser}} A_t \text{dot1} \cdot \text{dot2} \cdot \text{dot3} \cdot \text{dot4}}{4\pi^2 d_1^2 d_2^2} \quad (15)$$

es decir, en el código,

```
intensity = laser_intensity * area * (dot1*dot2*dot3*dot4)
          / (fourpi * d1s * d2s) ,
```

con `fourpi = 4.0 * torch.pi * torch.pi`.

Los dos $1/d^2$. Cada tramo de propagación libre reparte la energía emitida sobre una superficie que crece como el cuadrado de la distancia, así que la irradiancia que llega cae como $1/d^2$. Hay dos tramos (d_1 y d_2) y por tanto dos factores. Es importante notar que el código usa d_1^2 y d_2^2 **sin raíz** (`d1s` y `d2s` son las normas al cuadrado), lo que evita dos raíces cuadradas por elemento; las raíces d_1 y d_2 se calculan aparte solo para normalizar los vectores de los cosenos y para el tiempo de vuelo.

La constante $4\pi^2$. Cada reflexión difusa reparte la potencia incidente sobre un hemisferio completo, cuyo ángulo sólido es 2π sr; normalizar para que la energía se conserve introduce un factor $1/(2\pi)$ por reflexión. Como aquí hay **dos** eventos de emisión difusa (el punto iluminado del suelo y la faceta),

$$(2\pi)^2 = 4\pi^2 \approx 39,478. \quad (16)$$

Esta lectura es autoconsistente con la del *writeup* de referencia, que usa $8\pi^3 = (2\pi)^3$: allí hay **tres** emisiones difusas porque se modela también el tramo suelo→cámara como una reflexión. Aquí ese tramo no es una reflexión (el SPAD observa el parche directamente, sección 2.2), de ahí que sobre un factor 2π . La regla mnemotécnica es $(2\pi)^{\text{\#emisiones difusas}}$.

Salvedad 1 (La constante global *sí* afecta al CRB). Podría parecer que un factor multiplicativo global en $\mathbf{y}$ es irrelevante porque «se cancela». No es así. Con ruido de Poisson la varianza *es* la media, de modo que al escalar $\mathbf{y} \mapsto \alpha \mathbf{y}$ con el fondo b fijo, la información de Fisher (32) no escala de forma trivial y el CRB cambia. En el régimen dominado por señal ($y \gg b$) se tiene $\mathbf{I} \propto \alpha$ y por tanto $\sigma \propto \alpha^{-1/2}$: cuadruplicar la potencia del láser reduce las σ a la mitad. Las constantes radiométricas y `laser_intensity=1000` son, por tanto, parte del resultado y no adornos.

2.5. Área del triángulo e intensidad del láser

El factor A_t de (15) es el peso de la cuadratura (8): la contribución de un triángulo es proporcional a su elemento de superficie. El código lo calcula con la mitad de la norma del producto vectorial de dos aristas,

$$A_t = \frac{1}{2} \|(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)\|, \quad (17)$$

que es la fórmula exacta del área de un triángulo en $\mathbb{R}^3$ (`areas = 0.5 * norm(cross(...))`). Sumando sobre los 32 768 triángulos se recupera el área total de la faceta, $0,79251 \text{ m}^2 = 0,5 \times 1,585$, lo que confirma que la densificación conserva la geometría.

El factor $I_{\text{láser}} = \text{laser_intensity} = 1000$ fija la escala absoluta. Su papel es el de la salvedad 1: convierte el resultado geométrico en un número de cuentas esperadas.

2.6. Oclusión: derivación de x_{int}

Este es el paso menos evidente del código y el que aporta casi toda la información angular, así que conviene derivarlo con cuidado.

El problema. Un triángulo de la faceta (en $y > 0$) puede enviar luz a un píxel del suelo (en $y < 0$) solo si el segmento que los une pasa por el hueco de la esquina. La arista ocluyente está sobre $y = 0$ y tapa $x < 0$. Proyectado sobre el plano horizontal, el segmento cruza la recta $y = 0$ exactamente una vez, y la pregunta es *en qué x* .

La derivación. Sean (c_x, c_y) la proyección horizontal del centroide del triángulo y (q_x, q_y) la del píxel. La recta que los une, en forma $y = mx + b$, tiene pendiente y ordenada

$$m = \frac{c_y - q_y}{c_x - q_x}, \quad b = c_y - m c_x, \quad (18)$$

que es literalmente lo que hace el código:

```
denom = cx - cam_x; m = (cy - cam_y) / denom; b = cy - m * cx .
```

El cruce con $y = 0$ se obtiene resolviendo $0 = mx + b$:

$$\boxed{x_{\text{int}} = -\frac{b}{m}} \quad (x_{\text{int}} = -b / m). \quad (19)$$

Por qué $x_{\text{int}} > 0$ es el test correcto. Como el centroide está en $y > 0$ y el píxel en $y < 0$, el punto de cruce $(x_{\text{int}}, 0)$ está *dentro* del segmento, no en su prolongación. Por tanto:

- $x_{\text{int}} > 0$: el segmento atraviesa $y = 0$ del lado abierto de la esquina, luego el camino está libre y el par (triángulo, píxel) es visible;
- $x_{\text{int}} < 0$: el segmento intentaría atravesar la pared, luego el camino está bloqueado.

El código lo escribe como `noc = isfinite(xint) & (xint > 0)` (*no occlusion*). Es un modelo de oclusión **de medio plano**: la pared se supone infinita en $x < 0$ y de altura infinita, así que basta el test horizontal. Esa es la simplificación que hace que la oclusión salga gratis, sin trazar rayos contra ninguna geometría.

Las guardas numéricas de la versión GPU. Si el centroide y el píxel tienen la misma x , la recta es vertical y m diverge. El código lo evita con

```
valid_denom = abs(denom) > eps; m = where(valid_denom, (cy-cam_y)/denom, nan),
```

con `eps` = 10^{-12} , y después el `isfinite(xint)` descarta esos casos. Poner `nan` y filtrar es preferible a poner un número grande, porque un m enorme daría un x_{int} minúsculo de signo arbitrario, es decir, una decisión de visibilidad esencialmente aleatoria. En la pose verificada ningún elemento cae en este caso (0 máscaras activas, 0 x_{int} no finitos), pero la guarda evita un fallo silencioso.

Salvedad 2 (El test es por centroide: no hay visibilidad parcial). La visibilidad se decide por el *centroide* de cada triángulo: un triángulo está entero visible o entero oculto. La frontera real de la sombra corta triángulos por la mitad, y ese corte no se modela. El error que esto introduce es de orden «un triángulo» en la frontera, y es precisamente por eso que la densificación a 32768 triángulos (observación 3) importa: cuanto más finos los triángulos, mejor se aproxima la frontera. Tampoco hay supermuestreo dentro del píxel; el simulador antiguo tenía un parámetro `subpixel_dim` que el nuevo no usa.

La figura 4 muestra el campo x_{int} sobre la grilla y la máscara resultante: para el centroide de la faceta en $(1\text{ m}, 60^\circ)$, 802 de los 1024 píxeles ven la faceta, y la frontera $x_{\text{int}} = 0$ es una recta en el plano del suelo.

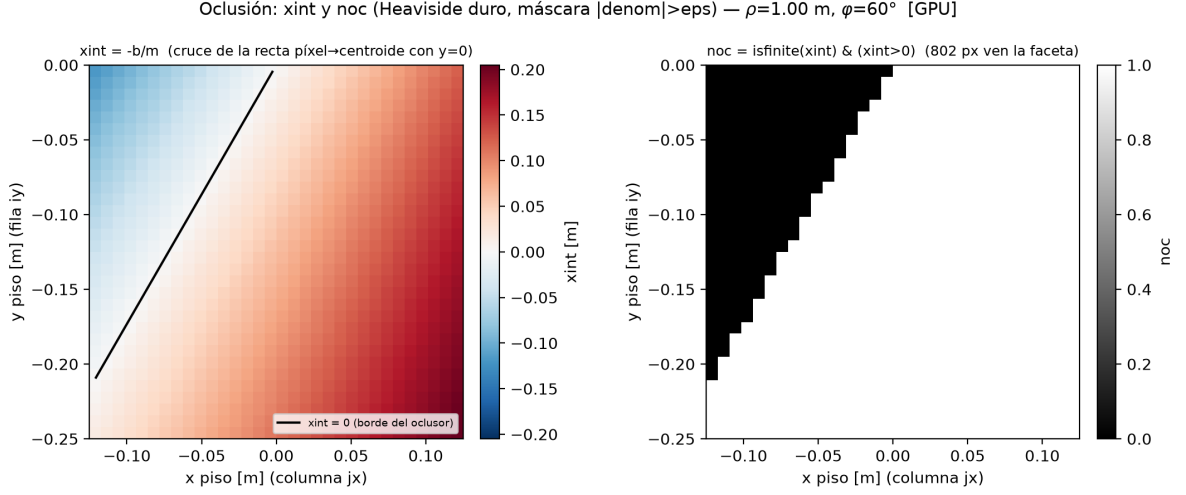


Figura 4: Izquierda: $x_{\text{int}} = -b/m$ por píxel, con la línea negra marcando $x_{\text{int}} = 0$ (el borde de la sombra del oclisor). Derecha: la máscara booleana noc , escalonada con el paso de la grilla porque la decisión es por píxel y por centroide. Esta discontinuidad es la fuente de la información angular y el principal obstáculo a la diferenciación.

2.7. Cuantización temporal y depósito

Tiempo de vuelo y bin. El tiempo que tarda la luz en recorrer el camino de dos rebotes es $t_0 = (d_1 + d_2)/c$, y el *bin* en que el SPAD lo registra es

$$j(t, p) = \left\lceil \frac{d_1(t) + d_2(t, p)}{c \Delta t} \right\rceil \quad (\text{arrival_bin} = \text{ceil}((d_1+d_2)/(C*\text{bin_size}))). \quad (20)$$

El $\lceil \cdot \rceil$ es el modelo del contador: un SPAD con *binning* de anchura Δt no informa del instante exacto de llegada, sino del intervalo $[(j-1)\Delta t, j\Delta t)$ en que cayó. Toda la masa del triángulo se deposita íntegra en ese *bin*; no se reparte entre *bins* vecinos. El código valida además que el índice caiga en rango: `valid = noc & (arrival_bin > 0) & (arrival_bin <= num_time_bins)`.

Cuántos bins hay: num_time_bins. El eje temporal se dimensiona a partir del camino más largo posible en la escena:

$$T = \left\lceil \frac{d_{\text{máx}}}{c \Delta t} \times 1,2 \right\rceil, \quad d_{\text{máx}} = \|\mathbf{p}_{\text{esc}}\| + \|\mathbf{p}_{\text{spad}} - \mathbf{p}_{\text{esc}}\|, \quad (21)$$

con $\mathbf{p}_{\text{esc}} = (x_{\text{máx}}, y_{\text{máx}}, z_{\text{máx}}) = (1, 5, 3, 3)$ la esquina más lejana del volumen de escena y $\mathbf{p}_{\text{spad}} = (-\text{FOV}/2, -\text{FOV}, 0) = (-0,125, -0,25, 0)$ la del parche observado. Numéricamente $d_{\text{máx}} = 9,2120$ m y

$$T = \left\lceil 9,2120/c / (3,9 \cdot 10^{-10}) \times 1,2 \right\rceil = 95.$$

El margen de 1,2 es un colchón del 20%: garantiza que ningún camino real se salga del cubo, aun con objetos colocados en los límites del volumen. Si faltara, los *bins* tardíos se descartarían por la máscara `valid` y se perdería energía en silencio.

Depósito disperso. El código no construye un tensor $T_{\Delta} \times P \times T$ (sería enorme), sino que aplana la salida y suma con un *scatter-add*:

$$\text{coord} = (j-1) \cdot P + \text{pixel_linear}[p], \quad \text{y_meas_vec.index_add}(0, \text{coord}, \text{intensity}). \quad (22)$$

El uso de `index_add_` en lugar de una asignación es *obligatorio*: muchísimos pares (triángulo, píxel) comparten el mismo `coord`, y lo que se quiere es la **suma** de sus contribuciones, que es exactamente la cuadratura (8). Una asignación se quedaría con una contribución arbitraria.

El índice `pixel_linear` y el `reshape` en orden Fortran. Este par de líneas es el que más confunde, así que lo desmenuzamos. Los centros de píxel se generan con `meshgrid(pixel_y, pixel_x, indexing="ij")` y se aplanan en orden C, de modo que el píxel k tiene coordenadas (x_j, y_i) con $i = k \text{ div } N$ y $j = k \text{ mód } N$. El código define

$$\text{pixel_linear}[k] = \underbrace{(k \text{ mód } N)}_j \cdot N + \underbrace{(k \text{ div } N)}_i = jN + i, \quad (23)$$

es decir, **transpose** el índice. Al final se hace `reshape((N,N,T), order="F")`, y en orden Fortran el índice plano f corresponde a (a, b, c) con $f = a + Nb + N^2c$. Sustituyendo (22): $c = j - 1$ (el *bin*, base cero) y $a + Nb = jN + i$, de donde $a = i$ y $b = j$. Por tanto

$$\mathbf{y}[a, b, c] \text{ con } a = \text{índice de píxel en } y, \quad b = \text{índice de píxel en } x, \quad c = j - 1. \quad (24)$$

La transposición de `pixel_linear` y el `order="F"` se **cancelan** entre sí y dejan el orden de ejes (`y_pixels`, `x_pixels`, `time_bins`) que el propio código declara en los atributos del HDF5. Verificado exhaustivamente con una malla de prueba $4 \times 4 \times 3$.

El `roll` de orientación. Por último, `simulation` aplica

$$\mathbf{y} \leftarrow \text{np.roll}(\mathbf{y}, \text{shift} = 1, \text{axis} = 1) \quad (\text{orient_transient_measurement}), \quad (25)$$

una permutación cíclica de una posición a lo largo del eje de píxeles en x . Es una corrección de orientación heredada, y comprobadamente activa: la salida real difiere de la no rotada. Su efecto visible es que la columna $j = N - 1$ pasa a ocupar $j = 0$, lo que en la figura 6 se aprecia como una columna «fuera de lugar» en el borde izquierdo. Ahora bien:

Proposición 2 (El `roll` no altera el CRB). *Sea π cualquier permutación de los índices de medición y sea $\mathbf{g}_q^\pi = \mathbf{g}_{\pi(q)}$. Entonces la información de Fisher de (32) cumple $\mathbf{I}[\mathbf{g}^\pi] = \mathbf{I}[\mathbf{g}]$, y por tanto el CRB, las σ y las elipses son idénticos.*

Demostración. $\mathbf{I}_{mk} = \sum_q \frac{1}{g_q} \partial_m g_q \partial_k g_q$ es una suma sobre *todos* los índices q . Como π es una biyección del conjunto de índices sobre sí mismo, reordenar los sumandos no cambia la suma. Además el fondo b es constante, luego $\mathbf{g}^\pi = \pi(\mathbf{y}) + b = \pi(\mathbf{y} + b)$ y la permutación conmuta con la suma del fondo. $\square$

Comprobado numéricamente: recalculando el CRB con el `roll` deshecho, la diferencia relativa máxima en $\mathbf{I}$ es $1,3 \cdot 10^{-16}$ en $(1 \text{ m}, 90^\circ)$ y $1,2 \cdot 10^{-15}$ en $(0,5 \text{ m}, 30^\circ)$, con $\Delta\sigma_\rho = 0$ exactamente. Es decir: el `roll` importa si se van a *mirar* las imágenes o reconstruir la escena, y es irrelevante para todo lo que sigue en estas notas.

Cuán cuantizado está el rango. Aquí hay que distinguir dos cosas que a menudo se confunden.

Para un punto fijo de la faceta, el *bin* de llegada apenas varía sobre la imagen: en la figura 5, el centroide produce solo **tres** valores enteros distintos (22, 23, 24) en los 1024 píxeles, porque d_2 recorre menos de 26 cm sobre todo el FOV. El corte 1D de la derecha muestra la escalera del $\lceil \cdot \rceil$ frente a la recta continua: mesetas largas separadas por saltos.

Para la faceta completa, en cambio, el transitorio de un solo píxel ocupa del orden de 15 a 21 *bins*, porque los 32 768 triángulos se reparten a lo largo de 1,585 m de altura y sus caminos $d_1 + d_2$ difieren mucho. Medido sobre la salida real:

ρ [m]	φ [°]	<i>bins</i> con señal	píxeles iluminados	<i>bins</i> por píxel (medio)
0.5	30	7–29	1016/1024	20.9
1.0	60	16–33	888/1024	15.9
1.0	90	17–33	640/1024	15.7
1.5	150	25–38	105/1024	12.7

Las dos afirmaciones son compatibles y ambas serán importantes: la primera explica por qué la derivada respecto de ρ es problemática (sección 5), y la segunda por qué a pesar de todo funciona.

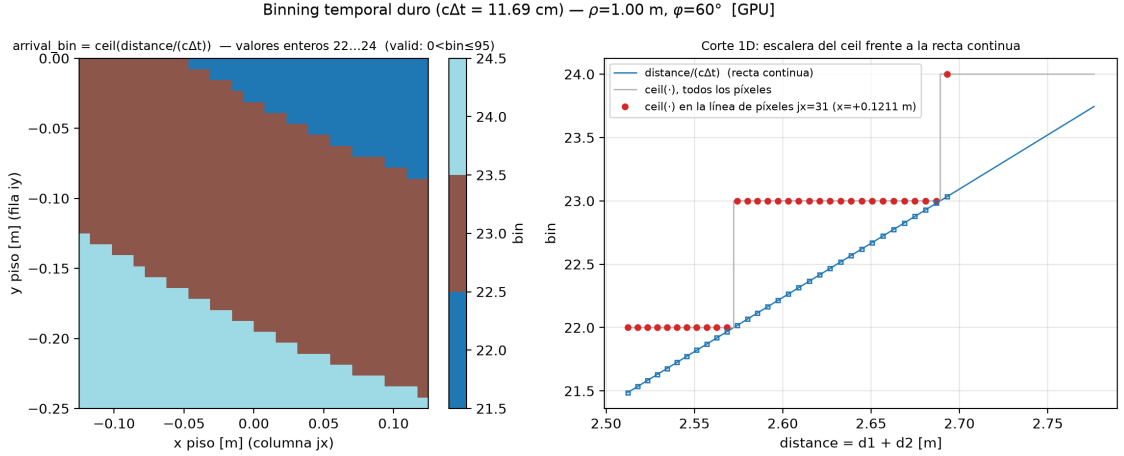


Figura 5: Cuantización temporal para el centroide de la faceta. Izquierda: el *bin* de llegada toma solo los valores 22, 23, 24 sobre la grilla entera. Derecha: la escalera del $\lceil \cdot \rceil$ contra la recta continua $(d_1 + d_2)/(c\Delta t)$, sobre la fila de píxeles $j = 31$.

2.8. El transitorio esperado

Reuniendo todo, el modelo directo completo es

$$y_{abc}(\psi) = \sum_{t=1}^{T_{\Delta}} \mathbf{1}[x_{\text{int}}(t, p) > 0] \mathbf{1}[j(t, p) = c + 1] \frac{I_{\text{láser}} A_t \prod_{r=1}^4 \max(0, \cos \theta_r)}{4\pi^2 d_1^2(t) d_2^2(t, p)} \quad (26)$$

donde el píxel p es el (a, b) de (24) tras el `roll`. El resultado es un cubo $32 \times 32 \times 95$, es decir 97 280 números.

La figura 6 muestra qué se ve: un mapa de la intensidad máxima por píxel, donde la sombra del oclisor aparece como una cuña oscura, y el histograma temporal de un píxel, con la deposición discreta por $\lceil \cdot \rceil$.

Observación 6 (Coste y validación de la vectorización). El bucle sobre `triangle_chunk_size=256` triángulos procesa un bloque contra los 1024 píxeles de golpe, en lugar de un triángulo por vez. Es puramente una reorganización aritmética: una réplica independiente en `numpy` del bucle fiel reproduce la salida de `simulator.simulation` con error máximo $5,6 \cdot 10^{-17}$ (relativo $3,1 \cdot 10^{-16}$), es decir, a precisión de máquina. El coste en CPU con `float64` es de unos 1,2–1,7 s por evaluación a 32×32 .

3. Del transitorio al modelo estadístico

3.1. Por qué Poisson

El transitorio (26) es una *tasa esperada*, no una medición. Un SPAD no mide intensidad: cuenta fotones individuales. La estadística de ese conteo es de Poisson por una razón fundamental: los fotones llegan como eventos independientes en el tiempo, con una tasa instantánea proporcional a la irradiancia, y el número de eventos de un proceso de Poisson en una ventana fija sigue una distribución de Poisson. Que el detector sea de conteo de fotón único hace que esta descripción sea exacta, no una aproximación de conveniencia: no hay un ruido gaussiano aditivo dominante que la enmascare.

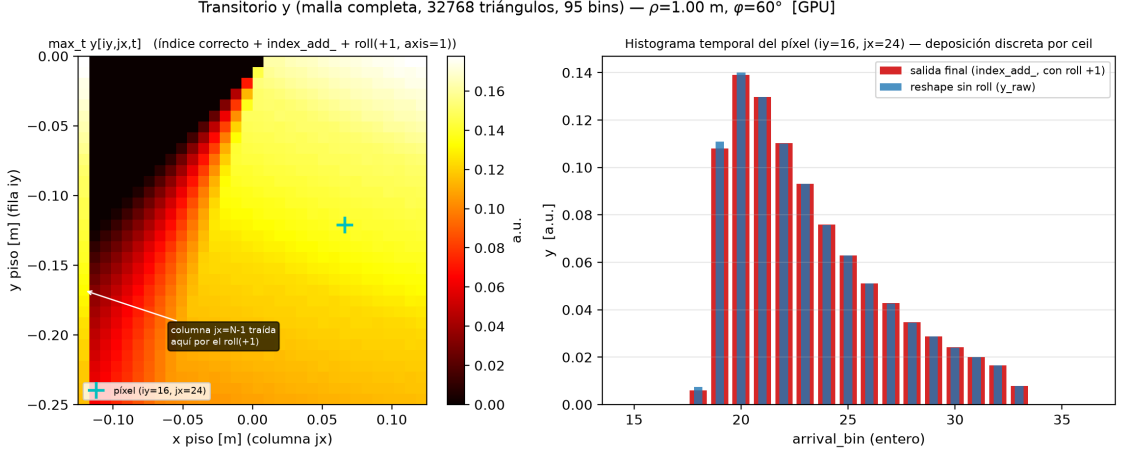


Figura 6: El transitorio esperado con la malla completa de 32 768 triángulos. Izquierda: máximo temporal por píxel; la cuña negra es la sombra del oclisor (comparar con la máscara de la figura 4) y la anotación señala la columna que el `roll` trae desde el otro extremo. Derecha: histograma temporal de un píxel, con y sin `roll` superpuestos; ocupa unos 16 *bins*.

La consecuencia crucial para el CRB es que en Poisson

$$\text{Var}(x_q) = \mathbb{E}[x_q] = g_q, \quad (27)$$

la varianza *es* la media. No hay un parámetro de ruido libre: el nivel de señal fija por sí solo la incertidumbre. Esto es lo que hace que las constantes radiométricas importen (salvedad 1) y lo que da al peso $1/g_q$ de la sección 4 su interpretación.

3.2. El fondo b

El modelo de medición es

$$g_q(\psi) = y_q(\psi) + b, \quad b = 0,1 \text{ cuentas/bin}, \quad (28)$$

para cada uno de los $q = 1, \dots, 97\,280$ índices (píxel, *bin*). En el código esto es `forward_g_polar` de `crb_polar_functions.py`, que construye

```
background = background_rate * ones_like(y_signal)
```

y devuelve `y_signal + background`. El valor 0,1 es la constante `BASELINE_BACKGROUND_RATE` de `crb_fd_baseline.py`.

Tiene dos justificaciones que se refuerzan:

- **Física:** un SPAD real tiene cuentas oscuras y recibe luz ambiente, así que ningún *bin* tiene tasa exactamente cero. Un fondo plano es el modelo más simple de ese piso.
- **Matemática:** sin b , los *bins* sin señal tendrían $g_q = 0$ y el peso $1/g_q$ de (32) divergería. Como la inmensa mayoría de los 97 280 índices no reciben señal —solo unos 15–21 *bins* de cada píxel iluminado— esto no es un detalle marginal sino la diferencia entre un problema bien planteado y una división por cero. Con $b = 0,1$, el piso `fisher_eps` = 10^{-12} de `fisher_poisson` nunca llega a actuar.

Conviene ser preciso sobre qué aporta b al CRB: los índices con $\partial g_q / \partial \psi = 0$ (los que nunca ven la faceta, para ninguna de las perturbaciones) contribuyen *cero* a la información, por muy grande que sea $1/g_q$. El fondo no añade información; lo que hace es **diluir** la de los *bins* que sí tienen señal, porque aumenta su varianza sin aumentar su sensibilidad. Subir b empeora el CRB, como debe ser.

3.3. El CRB no consume datos

Una confusión frecuente merece una advertencia explícita. El CRB **no usa mediciones ruidosas**. Se evalúa enteramente sobre el modelo esperado: el *pipeline* llama al simulador con `add_noise=False` (forzado, véase la sección 5.5), de modo que $\mathbf{y}$ es la tasa limpia y el «ruido» entra solo analíticamente, a través de la forma de la verosimilitud de Poisson. No se generan realizaciones, no se promedia sobre repeticiones y no hay semilla aleatoria. Por eso el cálculo es determinista y reproducible bit a bit.

3.4. Independencia entre índices

La verosimilitud de la sección siguiente supone que los 97 280 conteos son mutuamente independientes dado ψ . Para el conteo de fotones esto es razonable: distintos *bins* temporales son ventanas disjuntas de un proceso de Poisson, y distintos píxeles son detectores físicamente separados, de modo que los eventos no se comparten.

Salvedad 3 (Los límites de la independencia). La suposición ignora tres efectos reales. El *tiempo muerto* del SPAD: tras una detección el píxel queda ciego un tiempo, lo que correlaciona negativamente *bins* consecutivos y, en régimen de flujo alto, sesga el histograma hacia los *bins* tempranos (*pile-up*). La *diafonía* óptica y eléctrica entre píxeles vecinos. Y la *jitter* temporal del detector, que difumina la asignación de *bin* y correlaciona vecinos. Nada de esto está en el modelo, así que el CRB que sigue es el de un detector ideal: optimista frente a un sensor real.

4. La información de Fisher

4.1. Derivación

Con conteos de Poisson independientes, la verosimilitud de una realización $\mathbf{x}$ es $\prod_q e^{-g_q} g_q^{x_q} / x_q!$ y la log-verosimilitud

$$\ell(\psi) = \sum_q [x_q \ln g_q(\psi) - g_q(\psi) - \ln x_q!]. \quad (29)$$

El último término no depende de ψ y desaparece al derivar. Derivando respecto de la componente m -ésima del parámetro,

$$\frac{\partial \ell}{\partial \psi_m} = \sum_q \left(\frac{x_q}{g_q} - 1 \right) \frac{\partial g_q}{\partial \psi_m} = \sum_q \frac{x_q - g_q}{g_q} \frac{\partial g_q}{\partial \psi_m}. \quad (30)$$

Esta expresión es la *score*. Nótese de paso que $\mathbb{E}[\partial \ell / \partial \psi_m] = 0$ porque $\mathbb{E}[x_q] = g_q$: el estimador de máxima verosimilitud está bien centrado, como debe ser.

La información de Fisher es la covarianza de la *score*:

$$I_{mk} = \mathbb{E} \left[\frac{\partial \ell}{\partial \psi_m} \frac{\partial \ell}{\partial \psi_k} \right] = \sum_q \sum_{q'} \frac{\mathbb{E}[(x_q - g_q)(x_{q'} - g_{q'})]}{g_q g_{q'}} \frac{\partial g_q}{\partial \psi_m} \frac{\partial g_{q'}}{\partial \psi_k}. \quad (31)$$

Aquí es donde entra la independencia. Para $q \neq q'$ los conteos son independientes y ambos factores tienen media cero, luego $\mathbb{E}[(x_q - g_q)(x_{q'} - g_{q'})] = 0$: **todos los términos cruzados se anulan**. Para $q = q'$ queda la varianza, que por (27) vale g_q . Sustituyendo:

$$I_{mk}(\psi) = \sum_q \frac{1}{g_q(\psi)} \frac{\partial g_q}{\partial \psi_m} \frac{\partial g_q}{\partial \psi_k} \quad (32)$$

o, en forma matricial, con $\mathbf{J}_{qm} = \partial g_q / \partial \psi_m$ el jacobiano de tamaño $97\,280 \times 2$,

$$\mathbf{I} = \mathbf{J}^\top \text{diag}(1/g) \mathbf{J}, \quad (33)$$

que es literalmente `crb_polar_functions.fisher_poisson`:

```
g_safe = np.maximum(g, eps); return J.T @ ((1.0/g_safe)[: , None] * J) .
```


4.2. Cómo leerla

Cada índice de medición contribuye a $\mathbf{I}$ un término

$$\frac{(\partial g_q / \partial \psi_m)^2}{g_q} = \frac{(\text{sensibilidad})^2}{\text{varianza}},$$

que es la forma canónica de una relación señal–ruido. Un *bin* informa sobre ρ en la medida en que (i) su tasa esperada *cambia* al mover ρ y (ii) esa tasa no está ella misma dominada por fluctuación. Los dos extremos aclaran la intuición: un *bin* en el que la señal es enorme pero insensible a la pose no aporta nada; un *bin* muy sensible pero con tasa casi nula tampoco, porque su propio ruido de Poisson lo ahoga — salvo que g_q pequeño en el denominador lo amplifique, que es la razón por la que los *bins* del *borde* del pulso, donde g es bajo y la derivada alta, son los más informativos.

Dos propiedades estructurales que se usarán después:

- $\mathbf{I}$ es una **suma sobre índices de medición**. De ahí la aditividad que explica el escalado con la resolución (sección 8).
- $\mathbf{I}$ es simétrica y semidefinida positiva por construcción (es de la forma $\mathbf{B}^\top \mathbf{B}$ con $\mathbf{B} = \text{diag}(1/\sqrt{g})\mathbf{J}$), lo que garantiza que sus autovalores son ≥ 0 y que la elipse de la sección 7 está bien definida.

5. El jacobiano por diferencias finitas

Todo lo anterior necesita $\mathbf{J} = \partial \mathbf{g} / \partial \boldsymbol{\psi}$. El *forward* (26) contiene indicadores y un $[\cdot]$, así que no hay derivada en forma cerrada. El *pipeline* la aproxima numéricamente.

5.1. Diferencias centrales

Para cada componente m del parámetro,

$$\mathbf{J}_{:,m} \approx \frac{\mathbf{g}(\boldsymbol{\psi} + \delta_m \mathbf{e}_m) - \mathbf{g}(\boldsymbol{\psi} - \delta_m \mathbf{e}_m)}{2 \delta_m}, \quad \boldsymbol{\delta} = (\delta_\rho, \delta_\varphi) = (0,01 \text{ m}, 0,5^\circ), \quad (34)$$

implementado en `crb_polar_functions.finite_difference_jacobian_polar` y configurado por `BASELINE_FD_STEPS` en `crb_fd_baseline.py`. Cada pose cuesta $1 + 2 \cdot 2 = 5$ evaluaciones del *forward*: una central para $\mathbf{g}$ y cuatro perturbadas para las dos columnas de $\mathbf{J}$.

Por qué centrales y no unilaterales. Desarrollando en serie una función suave,

$$\frac{g(\psi + \delta) - g(\psi - \delta)}{2\delta} = g'(\psi) + \frac{\delta^2}{6} g'''(\psi) + O(\delta^4),$$

los términos de orden par se cancelan por simetría y el error es $O(\delta^2)$, frente al $O(\delta)$ de $(g(\psi + \delta) - g(\psi))/\delta$. Para el mismo δ la diferencia central es un orden más precisa, a costa de una evaluación extra por columna. Es la elección correcta aquí, donde el *forward* es caro pero no prohibitivo.

5.2. El problema honesto: el *forward* es escalonado

El desarrollo de Taylor anterior supone que g es suave, y **no lo es**. Tres operaciones lo rompen: el $[\cdot]$ del *binning* (20), el indicador de oclusión $\mathbf{1}[x_{\text{int}} > 0]$ (19) y los $\max(0, \cdot)$ de los cosenos (14). Las dos primeras son constantes a trozos: su derivada es cero casi en todas partes y no existe en un conjunto de saltos.

La escala del problema se cuantifica así. Un *bin* vale $c\Delta t = 11,69 \text{ cm}$ de camino (3). Cuánto hay que mover ρ para avanzar un *bin* depende de la sensibilidad del camino, $|\partial(d_1 + d_2)/\partial \rho|$, que medida en la pose $(1 \text{ m}, 90^\circ)$ vale 1,60 (y no 2: el tramo d_1 crece casi a razón de uno, pero d_2 crece

más despacio porque el parche observado no está en el origen sino desplazado a $(0, -0,125)$). Por tanto

$$\Delta\rho_{\text{bin}} = \frac{c \Delta t}{|\partial(d_1 + d_2)/\partial\rho|} = \frac{0,1169 \text{ m}}{1,60} = 7,3 \text{ cm}, \quad (35)$$

mientras que la diferencia central abarca $2\delta_\rho = 2 \text{ cm}$, es decir **el 27 % de un bin**. La consecuencia es directa: para un *triángulo aislado*, las dos evaluaciones $\mathbf{g}(\rho \pm \delta_\rho)$ caen casi siempre en el *mismo bin*, la diferencia es cero y la derivada estimada es 0; y cuando por casualidad cruzan un borde, salta una masa entera y la derivada estimada es $\propto 1/(2\delta_\rho)$. Ni una ni otra es la derivada de nada.

5.3. Por qué funciona a pesar de todo

Dos mecanismos de promediado rescatan el cálculo, y ambos son consecuencia de decisiones de modelado que ya justificamos por otras razones.

1. **La densificación a 32 768 triángulos.** Cada triángulo tiene su propio camino $d_1 + d_2$ y por tanto su propio umbral de salto. Los umbrales están repartidos casi uniformemente en el intervalo de un *bin*, porque la faceta mide 1,585 m de alto y los caminos se distribuyen de forma continua. Al mover ρ en δ_ρ , una *fracción* bien definida de los triángulos cruza su umbral, y esa fracción es aproximadamente proporcional a δ_ρ . La suma (8) convierte así una colección de escalones en una rampa: el agregado es aproximadamente lineal aunque cada sumando sea una escalera. Esto es exactamente lo que se ve en el panel central de la figura 7.
2. **La suma sobre índices de medición en (32).** La información no depende de la derivada de un píxel, sino de la suma de 97 280 contribuciones. Los residuos de discretización que quedan tienen signos distintos según el píxel y se cancelan parcialmente.

El panel derecho de la figura 7 pone a prueba esta explicación de la única manera honesta: variando δ_ρ y viendo si el resultado se mueve. Si el jacobiano fuese ruido de discretización, las σ saltarían de forma errática con el paso. Lo medido en $(1 \text{ m}, 90^\circ)$ es esto:

δ_ρ [mm]	2	5	10	20	40	80
σ_ρ [mm]	8.512	8.724	8.795	9.015	9.708	10.808
σ_φ [°]	2.5742	2.5741	2.5741	2.5752	2.5754	2.5576

La lectura es tranquilizadora en dos sentidos. σ_φ es *plana*, con variación total del 0.7 % sobre un rango de pasos de 40 : 1, como debe ser: δ_ρ solo afecta a la columna de ρ del jacobiano. Y σ_ρ crece **monótona y suavemente** con el paso —un 27 % al pasar de 2 a 80 mm—, que es la firma de un error de truncamiento $O(\delta^2)$ sobre una respuesta efectivamente suave, no la de un ruido de cuantización, que produciría saltos sin orden. El valor de referencia $\delta_\rho = 10 \text{ mm}$ queda a un 3 % del extrapolado a paso cero. Nada de esto *demuestra* que el modelo sea diferenciable; muestra que el promediado descrito arriba funciona lo bastante bien como para que el resultado no dependa de forma crítica de una elección arbitraria.

Una ilustración complementaria del mismo fenómeno está en la figura 8: si se sustituyera el $[\cdot]$ duro por un núcleo suave, la escalera de la faceta real se convertiría en la curva continua que el agregado ya aproxima de hecho. Es la idea que desarrolla la ruta analítica documentada en `reporte_consolidado.pdf`, y que reproduce a esta ruta FD dentro de un 10 %.

5.4. Por qué float64

El *baseline* fija `torch_dtype = torch.float64` en

`crb_fd_baseline.baseline_simulation_kwargs .`

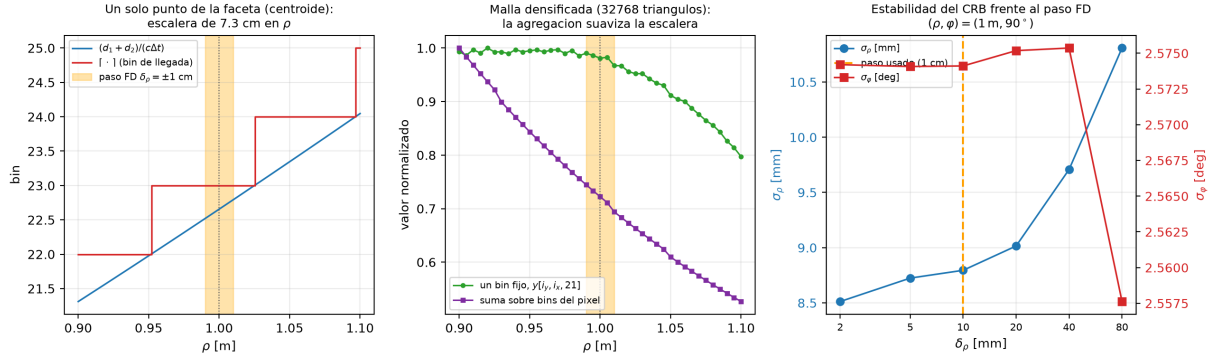


Figura 7: Por qué un paso de 1 cm funciona sobre un *forward* cuantizado en 7,3 cm de rango. Izquierda: para un punto fijo de la faceta el *bin* de llegada es una escalera, y la ventana del paso FD (banda amarilla) cabe holgadamente dentro de una meseta, de modo que la derivada de ese punto sería exactamente cero. Centro: con la malla densificada, tanto un *bin* fijo como la suma del píxel responden de forma suave a ρ ; esta es la razón por la que la diferencia finita tiene sentido. Derecha: σ_ρ y σ_φ frente al paso δ_ρ , en escala logarítmica; nótese la escala del eje de σ_φ , que abarca menos del 1%. Generada por docs/figs_forward/make_aux_figs.py.

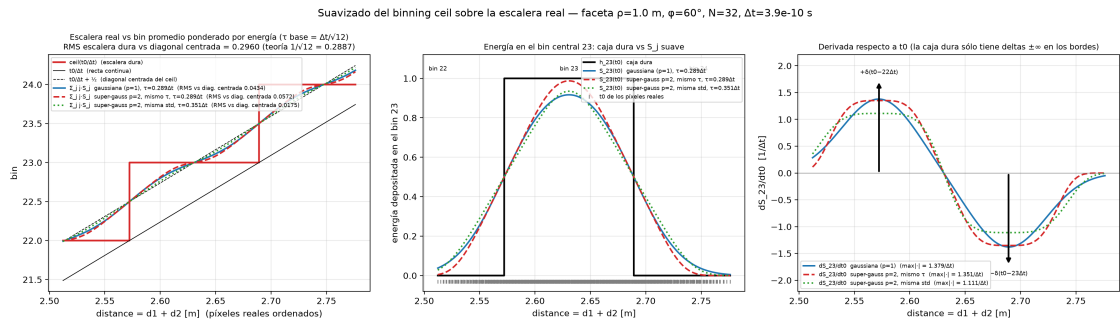


Figura 8: Suavizado del *binning* sobre la escalera de la faceta real, generado por plot_binning_smoothing.py: la cuantización dura frente a su versión con núcleo, que es la que tiene derivada clásica.

La razón es el numerador de (34): se resta un par de números casi iguales y se divide por un número pequeño. Con $\delta_\rho = 1$ cm la diferencia relativa entre $\mathbf{g}(\rho + \delta)$ y $\mathbf{g}(\rho - \delta)$ es del orden de 10^{-2} , y el épsilon relativo de `float32` es $\approx 1,2 \cdot 10^{-7}$: la cancelación catastrófica dejaría unos cinco dígitos significativos, que se propagarían elevados al cuadrado a **I**. Con `float64` ($\epsilon \approx 2,2 \cdot 10^{-16}$) el margen es amplísimo. El coste es un factor ~ 2 en tiempo, irrelevante frente al riesgo.

5.5. El adaptador de dos a tres valores

Hay una incompatibilidad de interfaz que conviene documentar aquí, porque es lo que conecta las secciones 2 y 4 en el código. La función

```
crb_polar_functions.simulate_facet_signal_expected
```

desempaqueta **tres** valores del *callable* de simulación,

```
_, y_signal, params = simulation_fn(**kwargs),
```

herencia del simulador antiguo del *notebook*, que devolvía la terna (**ruidoso**, **limpio**, **params**). El simulador nuevo devuelve **dos** valores, (**y_meas_vec_noisy**, **params**).

Se adaptó el lado del CRB, no el simulador. El adaptador

```
crb_fd_baseline.simulation_fn_for_crb
```

llama a `simulator.simulation` y re-emite su par como la terna (**y**, **y**, **params**). La operación es legítima porque el CRB siempre invoca el *forward* con `force_no_noise=True`, que fija `add_noise=False`; en ese caso el único transitorio devuelto es la tasa esperada sin ruido y puede ocupar ambas ranuras sin ambigüedad. El adaptador falla de forma explícita si se le pasa `add_noise=True`, para que nunca se pueda calcular una «Fisher» sobre una realización ruidosa. Con este diseño, `crb_polar_functions.py` no se modificó.

6. La cota de Cramér–Rao

6.1. El enunciado y su alcance

La desigualdad de Cramér–Rao dice que, para cualquier estimador $\hat{\psi}(x)$ **insesgado** de ψ ,

$$\text{Cov}(\hat{\psi}) \succeq \mathbf{I}(\psi)^{-1}, \quad (36)$$

donde $\succeq$ significa que la diferencia es semidefinida positiva. Tomando elementos diagonales, $\text{Var}(\hat{\rho}) \geq [\mathbf{I}^{-1}]_{11}$ y análogamente para φ . El *pipeline* define

$$\mathbf{C} = \mathbf{I}^{-1}, \quad \sigma_\rho = \sqrt{C_{11}}, \quad \sigma_\varphi = \sqrt{C_{22}}, \quad \sigma_{\text{tang}} = \rho \sigma_\varphi, \quad (37)$$

en `compute_crb_polar` y `_crb_sigmas_from_matrix`. El error tangencial $\rho \sigma_\varphi$ convierte la incertidumbre angular a metros sobre el arco, para poderla comparar con σ_ρ en las mismas unidades.

La inversión es una pseudoinversa. El código usa `CRB = np.linalg.pinv(I)` y no `inv`. La razón es robustez: si en alguna pose una dirección del espacio de parámetros no fuera observable, **I** sería singular y `inv` lanzaría una excepción, abortando la rejilla entera. La pseudoinversa devuelve el mejor sustituto (anulando la dirección no informativa) y permite detectar el problema mirando las σ . En la rejilla *baseline* **I** está bien condicionada en las 15 poses y `pinv` coincide con `inv`.

Qué no dice la cota. Tres precisiones que evitan sobreinterpretarla:

- Es una **cota inferior**, no el error de un estimador concreto. Ningún estimador tiene que alcanzarla; el de máxima verosimilitud la alcanza asintóticamente en muchos casos regulares, pero aquí el *forward* es escalonado y esa garantía no aplica sin más.
- Vale para estimadores **insesgados**. Un estimador sesgado puede tener varianza menor (el caso extremo, $\hat{\psi} = \text{constante}$, tiene varianza cero), a costa del sesgo.
- Es **local**: **I** se evalúa en un ψ concreto y describe la curvatura de la verosimilitud ahí. No dice nada sobre ambigüedades globales, por ejemplo dos poses muy separadas que produzcan transitorios parecidos.

6.2. La correlación entre ρ y φ

El elemento fuera de la diagonal es tan informativo como los de la diagonal, y se reporta como

$$r_{\rho\varphi} = \frac{C_{12}}{\sqrt{C_{11}C_{22}}} \quad (\text{columna } \texttt{corr} \text{ de la tabla } \textit{baseline}). \quad (38)$$

Por qué $I_{\rho\varphi} \neq 0$. Mirando (32), $I_{\rho\varphi} = \sum_q g_q^{-1} (\partial_\rho g_q)(\partial_\varphi g_q)$ se anula solo si ninguna medición es sensible a ambos parámetros a la vez. Aquí ocurre lo contrario, por dos vías:

- el **camino** $d_1 + d_2$ depende de la posición $(\rho \cos \varphi, \rho \sin \varphi)$, es decir de *ambas* coordenadas: girar la faceta también cambia su distancia al parche observado, que no está centrado en el origen sino en $(0, -0,125)$;
- la **oclusión** x_{int} (19) depende de la posición completa del centroide, así que mover ρ desplaza también la frontera de la sombra, que es el mecanismo por el que se mide φ .

Qué significa el signo. Todas las correlaciones de la rejilla *baseline* son **negativas**. Un $r_{\rho\varphi} < 0$ dice que los errores están anticorrelacionados: una sobreestimación de ρ tiende a acompañarse de una subestimación de φ . La lectura física es que existe una **dirección de compensación** en el plano (ρ, φ) a lo largo de la cual el transitorio cambia poco: alejar la faceta y girarla hacia el hueco produce, hasta primer orden, un camino y una sombra parecidos. El estimador puede distinguir bien un desplazamiento perpendicular a esa dirección y mal uno paralelo a ella; de ahí que las elipses de la sección 7 salgan alargadas e inclinadas.

Los valores medidos. La correlación no es uniforme sobre la rejilla: es más fuerte de cerca y con la faceta hacia el borde iluminado, y más débil de lejos.

$r_{\rho\varphi}$	$\varphi = 30^\circ$	$\varphi = 90^\circ$	$\varphi = 150^\circ$
$\rho = 0,5 \text{ m}$	-0,731	-0,435	-0,403
$\rho = 1,0 \text{ m}$	-0,564	-0,283	-0,256
$\rho = 1,5 \text{ m}$	-0,452	-0,209	-0,184

El recorrido completo va de -0,731 en $(0,5 \text{ m}, 30^\circ)$ a -0,184 en $(1,5 \text{ m}, 150^\circ)$; la tabla 2 da las 15 poses.

Observación 7 (Independientes como coordenadas, dependientes como estimadores). ρ y φ son coordenadas independientes: se pueden fijar por separado y nada las liga *a priori*. Lo que está correlacionado son los **errores de sus estimaciones**, porque los datos no las separan limpiamente. Son dos afirmaciones sobre objetos distintos: una sobre la parametrización, otra sobre la geometría de la verosimilitud.

Observación 8 (Las σ son conjuntas, no condicionales). Los σ_ρ y σ_φ de (37) salen de invertir la matriz 2×2 *completa*: son las incertidumbres de estimar los dos parámetros *a la vez*. La incertidumbre *condicional* (conociendo φ de antemano) sería $1/\sqrt{I_{\rho\rho}}$, y la relación entre ambas es

$$\sigma_\rho = \frac{1}{\sqrt{I_{\rho\rho}}} \cdot \frac{1}{\sqrt{1-r_I^2}}, \quad r_I = \frac{I_{\rho\varphi}}{\sqrt{I_{\rho\rho}I_{\varphi\varphi}}},$$

de modo que la incertidumbre conjunta es siempre **mayor** o igual que la condicional, y el factor de inflación crece al acercarse la correlación a ± 1 . Estimar un parámetro extra nunca sale gratis. Este es el mismo mecanismo por el que, en la ruta antigua de tres parámetros, añadir la altura h ensanchaba todas las burbujas.

7. De la matriz a la elipse

7.1. La región de confianza es una elipse

Dada la matriz de covarianza $\mathbf{C}$ (aquí el CRB), el conjunto

$$\mathcal{E}_k = \{\boldsymbol{\delta} \in \mathbb{R}^2 : \boldsymbol{\delta}^\top \mathbf{C}^{-1} \boldsymbol{\delta} \leq k^2\} \quad (39)$$

es un elipsoide (en dimensión 2, una elipse) centrado en el origen. Su frontera se parametriza de forma explícita mediante la descomposición espectral. Como $\mathbf{C}$ es simétrica y definida positiva, existe

$$\mathbf{C} = \mathbf{V} \boldsymbol{\Lambda} \mathbf{V}^\top, \quad \boldsymbol{\Lambda} = \text{diag}(\lambda_1, \lambda_2), \quad \mathbf{V} \text{ ortogonal}, \quad (40)$$

y sustituyendo $\boldsymbol{\delta} = \mathbf{V} \boldsymbol{\Lambda}^{1/2} \mathbf{u}$ en (39) la condición se vuelve $\|\mathbf{u}\| \leq k$. Por tanto la frontera es la imagen de un círculo de radio k :

$$\boldsymbol{\delta}(\vartheta) = k \mathbf{V} \boldsymbol{\Lambda}^{1/2} \begin{pmatrix} \cos \vartheta \\ \sin \vartheta \end{pmatrix}, \quad \vartheta \in [0, 2\pi), \quad (41)$$

es decir: **semiejes $k\sqrt{\lambda_i}$, orientados según los autovectores de $\mathbf{C}$** . Esto es exactamente `crb_region_in_polar_parameters`:

```
eigenvalues, eigenvectors = np.linalg.eigh(Sigma); axes = k *
    np.sqrt(eigenvalues);
ellipse = center[:,None] + eigenvectors @ np.diag(axes) @ circle,
```

con `circle` el círculo unidad muestreado en `n_points=300` y `np.maximum(eigenvalues, 0.0)` como guarda contra autovalores ligerísimamente negativos por redondeo.

7.2. Por qué $k = 3$

Si los errores fueran gaussianos de covarianza $\mathbf{C}$, la forma cuadrática $\boldsymbol{\delta}^\top \mathbf{C}^{-1} \boldsymbol{\delta}$ seguiría una χ^2 con 2 grados de libertad, cuya función de distribución es $F(u) = 1 - e^{-u/2}$. Con $k = 3$,

$$\Pr(\boldsymbol{\delta} \in \mathcal{E}_3) = 1 - e^{-k^2/2} = 1 - e^{-4.5} = 0.98889, \quad (42)$$

es decir, la región $\mathcal{E}_3$ contiene el 98.9 % de la masa de probabilidad.

Observación 9 (La regla de 3σ en 2D no es la de 1D). En una dimensión, $\pm 3\sigma$ cubre el 99.73 %. En dos dimensiones la misma k cubre solo el 98.89 %, porque hay «más sitio» fuera: la masa de probabilidad se reparte en un anillo cuyo perímetro crece con el radio. Conviene citar el número correcto, 98.9 %, y no el de 1D.

7.3. La inclinación es la correlación

La orientación de la elipse y la correlación de la sección 6.2 son la misma información vista de dos formas. Si $r_{\rho\varphi} = 0$, la matriz $\mathbf{C}$ es diagonal, sus autovectores son los ejes coordenados y la elipse sale **alineada** con ellos. Si $r_{\rho\varphi} \neq 0$, los autovectores giran; el ángulo de giro satisface

$$\tan(2\alpha) = \frac{2C_{12}}{C_{11} - C_{22}}, \quad (43)$$

y el signo de C_{12} determina hacia qué lado. Con $C_{12} < 0$ en toda la rejilla, el eje mayor de las elipses apunta en la dirección de compensación « ρ arriba, φ abajo»: mirar la inclinación de una burbuja es leer directamente qué combinación de rango y ángulo el sensor no consigue separar.

7.4. Por qué se dibuja en el espacio de parámetros

Hay dos formas de representar la región, y el *pipeline* usa deliberadamente la primera.

Burbujas: la región en el plano (ρ, φ) . Es (41) tal cual, centrada en (ρ_0, φ_0) . Se elige esta porque **el CRB vive en el espacio de parámetros**: es una cota sobre la covarianza de $\hat{\psi} = (\hat{\rho}, \hat{\varphi})$, y sus semiejes son literalmente las incertidumbres que se quieren leer. No hay ninguna transformación intermedia que pueda introducir error o confundir la interpretación.

Gotas: el empuje al plano físico xy . La alternativa es transportar la región a la posición física mediante la linealización de $(x, y) = (\rho \cos \varphi, \rho \sin \varphi)$:

$$\mathbf{A} = \frac{\partial(x, y)}{\partial(\rho, \varphi)} = \begin{pmatrix} \cos \varphi & -\rho \sin \varphi \\ \sin \varphi & \rho \cos \varphi \end{pmatrix}, \quad \Sigma_{xy} = \mathbf{A} \mathbf{C} \mathbf{A}^\top, \quad (44)$$

que es lo que implementa `crb_region_physical_xy_to_polar` (existe en el módulo y está disponible con `plot_crb_regions_polar(..., use_physical_region=True)`). Tiene dos inconvenientes. Primero, (44) es una **aproximación de primer orden**: la aplicación polar→cartesiana no es lineal, así que la imagen exacta de una elipse no es una elipse, y cuanto mayor sea $\rho \sigma_\varphi$ frente a ρ más se deforma. Segundo, al reconvertir a polares para dibujar sobre el eje polar, el contorno se curva y adopta la forma de gota que motivó abandonar esta representación: visualmente sugiere una asimetría que no está en el CRB, sino en el cambio de coordenadas.

El *pipeline* usa por tanto

```
plot_crb_regions_polar(results, use_physical_region=False, rlim=2.0, ...)
```

que selecciona las claves `rho_curve_param` y `phi_curve_param`, y dibuja sobre un eje polar con `ax.plot(phi_curve, rho_curve)`: φ como coordenada angular y ρ como radial. El estilo lo fija `_style_polar_crb_ax`: medio plano $\theta \in [0, 180^\circ]$, cero a la derecha (`set_theta_zero_location("E")`), sentido antihorario, etiquetas radiales a dos decimales y fuentes reducidas.

Observación 10 (Por qué las burbujas se ven «arqueadas»). Las elipses de la figura 10 no son elipses perfectas en la página, sino curvas arqueadas. No es una deformación del CRB: es que se está dibujando una elipse del plano cartesiano (φ, ρ) sobre unos *ejes polares*. Un segmento de ρ constante es un arco de circunferencia en el papel, así que la elipse aparece curvada. Es una consecuencia puramente gráfica de haber elegido un eje polar para que la posición de cada burbuja coincida con la dirección física de la pose.

Salvedad 4 (Unidades mixtas en el plano de la elipse). El plano (ρ, φ) mezcla metros y radianes, de modo que «semieje» y «ángulo de inclinación» dependen de la escala relativa que se elija entre los dos ejes. Los autovalores de (40) no son invariantes geométricos; cambiar φ de radianes a

grados cambiaría los semiejes y la inclinación. No es un error, es una convención: mientras se aplique la misma a todas las poses y a todos los métodos, las comparaciones son válidas. Lo que sí es invariante son las cantidades escalares σ_ρ y σ_φ por separado, y su combinación en unidades homogéneas $\rho \sigma_\varphi$.

8. El *baseline* de 32×32

8.1. Cómo escala la precisión con la resolución

De la estructura de (32) sale una predicción cuantitativa. $\mathbf{I}$ es una suma sobre índices de medición, y los índices son (píxel, *bin*). Al pasar de $N \times N$ a $2N \times 2N$ píxeles con el mismo FOV, el número de píxeles se cuadruplica. La pregunta es qué le pasa a la contribución de cada píxel.

Aquí hay un detalle del modelo que conviene explicitar: la intensidad (15) **no lleva ningún factor de área de píxel**. El simulador *muestrea* la radiancia del suelo en el centro de cada píxel; no la integra sobre su huella. Por tanto, al refinar la grilla, cada píxel conserva aproximadamente su valor esperado y simplemente hay cuatro veces más de ellos. Entonces

$$\mathbf{I} \propto N_{\text{px}} = N^2 \quad \implies \quad \sigma \propto \frac{1}{\sqrt{N_{\text{px}}}} = \frac{1}{N}. \quad (45)$$

Salvedad 5 (Este escalado es del modelo, no de la física). Un sensor real con FOV fijo recoge un flujo de fotones fijo; subdividirlo en más píxeles *reparte* esos fotones, y la información total se mantendría aproximadamente constante. La ley (45) aparece porque el modelo puntual multiplica el número de muestras sin dividir la señal entre ellas. Es consistente y útil para comparar configuraciones, pero no debe leerse como «poner más píxeles mejora la precisión física»: describe cómo cambia el CRB de *este* modelo. Para modelar el reparto habría que multiplicar (15) por la huella del píxel, $(\text{FOV}/N)^2$.

El barrido de `sr_crb.py` (que usa `utils/crb_utils.py` sobre el mismo *forward* y el mismo estimador) confirma (45) con precisión notable. En la pose $(\rho, \varphi) = (1 \text{ m}, 90^\circ)$:

Tabla 1: Barrido de resolución en $(1 \text{ m}, 90^\circ)$, de `sr_crb.py -pixel-dims 8,16,32,64`. La última columna compara con la predicción $\sigma \propto 1/N$ tomando $N = 8$ como referencia.

N	σ_ρ [mm]	σ_φ [°]	$\sigma_\rho(8)/\sigma_\rho(N)$	$N/8$
8	35.11	10.380	1.00	1
16	17.55	5.092	2.00	2
32	8.79	2.574	3.99	4
64	4.40	1.294	7.98	8

El acuerdo es casi exacto: al octuplicar N , σ_ρ cae un factor 7,98 frente al 8 predicho. Eso confirma tanto la aditividad de (32) como la ausencia de factor de área de píxel, y de paso da una comprobación de consistencia útil sobre el jacobiano FD: un artefacto de discretización no escalaría con esta limpieza. Las curvas completas en rango y en ángulo están en la figura 9.

8.2. La rejilla *baseline*

Con la decisión $N = 32$ y el resto de la configuración fijada como la usaba el repositorio (FOV 0,25 m, $\Delta t = 3,9 \cdot 10^{-10}$ s, $I_{\text{láser}} = 1000$, paredes ocultas, sin ruido, `facet.obj` con $w = 0,5$ m, $b = 0,1$, pasos FD $(0,01 \text{ m}, 0,5^\circ)$), el *script* `compute_crb_fd_baseline.py` evalúa las 15 poses $\rho \in \{0,5, 1,0, 1,5\} \text{ m}$, $\varphi \in \{30, 60, 90, 120, 150\}^\circ$.

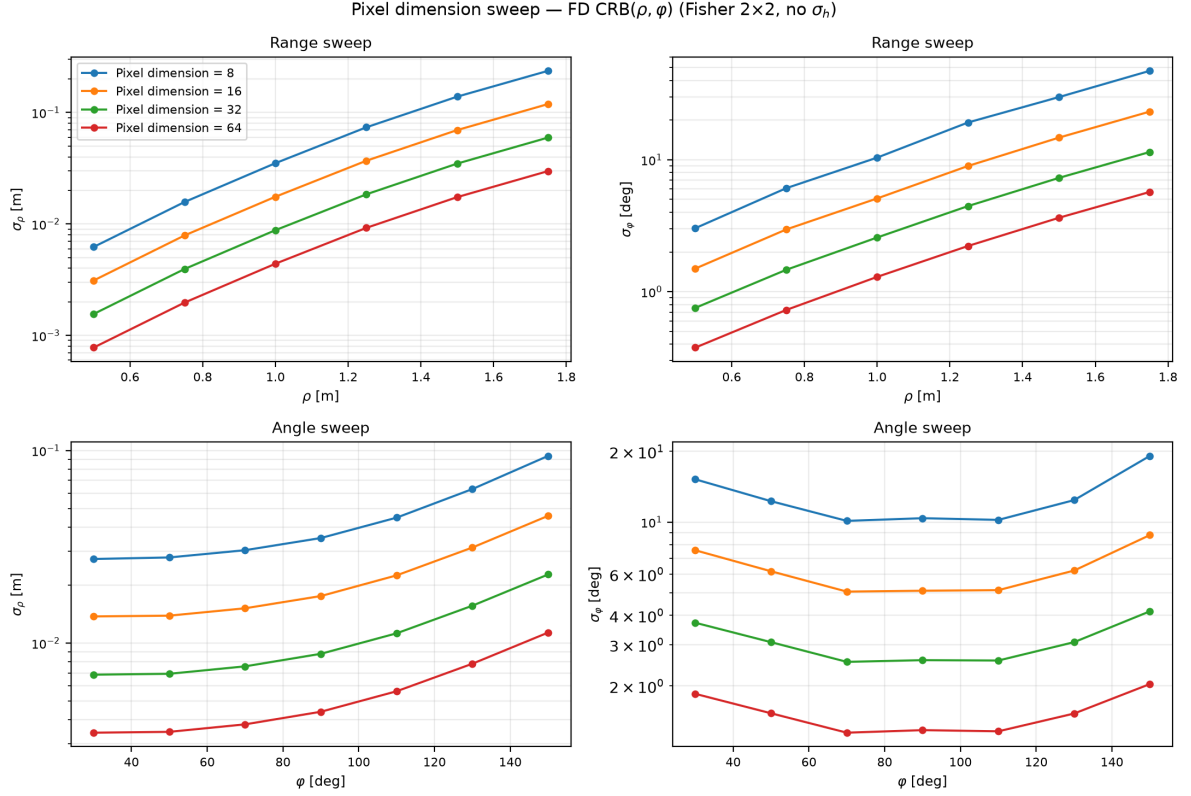


Figura 9: Barrido de resolución con `sr_crb.py` -pixel-dims 8,16,32,64. Arriba: σ_ρ y σ_φ frente a ρ con $\varphi = 90^\circ$ fijo. Abajo: frente a φ con $\rho = 1$ m fijo. Las cuatro curvas de cada panel son paralelas en escala logarítmica y equiespaciadas por factores de 2, que es (45). El crecimiento de σ_ρ con ρ es aproximadamente exponencial en el rango explorado, consecuencia del $1/(d_1^2 d_2^2)$ de (15).

Tabla 2: Rejilla *baseline*: CRB(ρ, φ) por diferencias finitas sobre `simulator.py` con grilla 32×32 y Fisher 2×2 . No hay columna σ_h : no está definida (sección 10).

ρ [m]	φ [°]	σ_ρ [m]	σ_φ [°]	$\rho \sigma_\varphi$ [m]	$r_{\rho\varphi}$
0.5	30	0.00136	1.0034	0.00876	-0,731
0.5	60	0.00135	0.8721	0.00761	-0,583
0.5	90	0.00156	0.7535	0.00658	-0,435
0.5	120	0.00222	0.8106	0.00707	-0,402
0.5	150	0.00370	1.0872	0.00949	-0,403
1.0	30	0.00685	3.7173	0.06488	-0,564
1.0	60	0.00716	2.7136	0.04736	-0,379
1.0	90	0.00879	2.5741	0.04493	-0,283
1.0	120	0.01318	2.6703	0.04661	-0,250
1.0	150	0.02271	4.1593	0.07259	-0,256
1.5	30	0.02566	10.7727	0.28203	-0,452
1.5	60	0.02775	7.3200	0.19164	-0,280
1.5	90	0.03482	7.2896	0.19084	-0,209
1.5	120	0.05176	7.3811	0.19324	-0,191
1.5	150	0.09031	11.6853	0.30592	-0,184

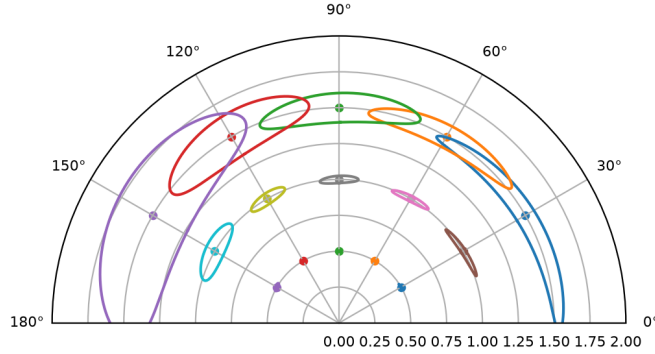


Figura 10: Regiones 3σ del $\text{CRB}(\rho, \varphi)$ por diferencias finitas sobre `simulator.py`, grilla 32×32 . Burbujas en el espacio de parámetros dibujadas sobre ejes polares, con `rlim= 2,0`. Se regenera con `python3 compute_crb_fd_baseline.py`.

8.3. Cómo leer la rejilla

Cuatro tendencias, todas explicables con lo construido hasta aquí:

- **La precisión cae muy rápido con la distancia.** De $\rho = 0,5$ a $\rho = 1,5$ m, σ_ρ crece entre 19 y 25 veces según el ángulo (de 1,36 a 25,7 mm en 30° , de 3,70 a 90,3 mm en 150°). El motivo está en (15): la señal decae como $1/(d_1^2 d_2^2)$, y con varianza igual a la media, menos señal es directamente menos información.
- **El ángulo se estima peor en los extremos.** σ_φ es mínima cerca de $\varphi = 90^\circ$ y máxima en 30° y 150° . En 150° la faceta está muy ocluida por la arista (en la tabla de la sección 2.7, solo 105 de 1024 píxeles reciben algo en (1,5, 150°)), y la información angular la aporta precisamente la frontera de la sombra.
- **El error tangencial domina el radial** por un factor entre 3,4 y 6,5 en todas las poses. La cámara de esquina localiza el rango mucho mejor que el azimuth: el tiempo de vuelo mide ρ de forma directa, mientras que φ se infiere indirectamente de la geometría de la sombra. Es la anisotropía que produce las elipses muy alargadas.
- **Las correlaciones son todas negativas** (sección 6.2), lo que inclina todas las burbujas en el mismo sentido. Su magnitud decrece monótonamente con ρ y, en φ , tiene su mínimo cerca de 120° y vuelve a crecer ligeramente en 150° .

Comparando con la referencia FD anterior a 64×64 , la degradación por bajar a 32×32 es un factor $\approx 1,9$ – $2,4$ en σ_ρ y $\approx 1,6$ – $1,9$ en σ_φ sobre la rejilla completa, y exactamente 2,00 y 1,99 en la pose (1 m, 90°) de la tabla 1: coherente con (45), que predice 2.

9. Mapa del código

Archivo	Función	Etapas de estas notas
simulator.py	<code>_transform_object_mesh</code> <code>simulate_transient_torch</code> <code>simulation</code> <code>orient_transient_measurement</code>	colocación de la faceta (§2.1) distancias, cosenos, atenuación, oclusión y <i>binning</i> (§2.2–§2.7) orquestración, <code>num_time_bins</code> , ruido opcional (§2.7) el roll de (25) (§2.7 y prop. 2)
utils/ densify_mesh.py	<code>densify_mesh_if_needed</code>	cuadratura de superficie (§2.1)
crb_fd_baseline.py	<code>simulation_fn_for_crb</code> <code>baseline_simulation_kwargs</code> , <code>BASELINE_*</code>	adaptador de $2 \rightarrow 3$ valores (§5.5) configuración <i>baseline</i> (§8)
crb_polar_ functions.py	<code>forward_g_polar</code> <code>finite_difference_jacobian_polar</code> <code>fisher_poisson</code> <code>compute_crb_polar</code> <code>crb_region_in_polar_parameters</code> <code>crb_region_physical_xy_to_polar</code> <code>plot_crb_regions_polar</code> , <code>_style_polar_crb_ax</code>	$\mathbf{g} = \mathbf{y} + b$, ec. (28) (§3) jacobiano FD, ec. (34) (§5) $\mathbf{I} = \mathbf{J}^\top \text{diag}(1/\mathbf{g})\mathbf{J}$, ec. (33) (§4) $\mathbf{C} = \mathbf{I}^+$, las σ y la correlación (§6) elipse, ec. (41) (§7) empuje a xy , ec. (44) (§7) dibujo de las burbujas (§7)
compute_crb_ fd_baseline.py	<code>main</code>	rejilla de 15 poses y figura 10 (§8)
sr_crb.py, utils/crb_utils.py	<code>run_sweep</code> , <code>plot_range_angle_sweep</code>	barrido de resolución, tabla 1 (§8)
plot_simulator_ components.py	—	figuras 2, 3, 4, 5 y 6
plot_binning_ smoothing.py	—	figura 8
docs/figs_forward/ make_aux_figs.py	—	figuras 1 y 7

10. Limitaciones y salvedades

Recogemos aquí, en un solo lugar, todo lo que el *pipeline* no hace, para que ningún número de estas notas se lea con más autoridad de la que tiene.

1. **No hay altura: el CRB es de dos parámetros y no existe σ_h .** El simulador escala la malla con (9) y **nunca lee una clave h** . La altura de la faceta es un resultado de la geometría del `.obj` y de w , no una entrada controlable, así que no existe $\partial \mathbf{g} / \partial h$ y la Fisher es 2×2 . El *pipeline* lo impone con `estimate_height=False`. Recuperar σ_h exigiría *añadir* una entrada de altura al simulador, lo que sería un cambio de diseño y no una adaptación. Toda la familia de resultados «CRB(ρ, φ, h) / Fisher 3×3 » que aparecía en versiones anteriores del estudio ya no es calculable.
2. **El jacobiano es una diferencia finita sobre un *forward* escalonado (§5).** Funciona por promediado, no porque el modelo sea diferenciable; el resultado tiene una dependencia residual del paso y no admite las garantías habituales de convergencia. La alternativa con derivadas exactas es la ruta analítica documentada en `docs/reporte_consolidado.pdf`, que suaviza `[·]`, el indicador y los `máx(0, ·)`, deriva en `sympy` y reproduce a esta ruta dentro de un 10 % en ambos ejes.
3. **La oclusión se decide por centroide** y con una pared de medio plano infinito (§2.6, salvedad 2). No hay visibilidad parcial de triángulos ni geometría real del ocluidor.

4. **No hay supermuestreo dentro del píxel.** La radiancia se evalúa en el centro del píxel, sin integrar sobre su huella, con las dos consecuencias de la salvedad 5: los bordes de sombra se resuelven al límite de la grilla y el escalado con N es el del modelo, no el físico.
5. **La suma sobre triángulos es una cuadratura** (§2.1). Con 32 768 triángulos el error es pequeño, pero no nulo, y depende de la pose.
6. **El modelo de detector es ideal:** Poisson independiente, sin tiempo muerto, *pile-up*, diafonía ni *jitter* (§3). El CRB resultante es optimista frente a un SPAD real.
7. **El CRB es una cota local para estimadores insesgados** (§6). No es el error de ningún estimador concreto y no captura ambigüedades globales.
8. **El plano de la elipse tiene unidades mixtas** (salvedad 4): semiejes e inclinación son convenciones consistentes, no invariantes geométricos.
9. **Detalles heredados que conviene no idealizar:** el *pitch* de 1,57 en lugar de $\pi/2$ (obs. 5), la cota de altura inerte (obs. 4) y el *roll* de orientación, que es una corrección empírica de índice — inofensiva para el CRB por la proposición 2, pero no una operación físicamente motivada.